PlanLi Schedule

Product Requirements Document — AI-Based Medical Scheduling System

Version 1.0 | March 2026 | CONFIDENTIAL — INTERNAL USE ONLY

Document	Product Requirements Document (PRD)	**Version**	1.0 — March 2026
Product	PlanLi Schedule — AI Medical Scheduler	**Status**	In Development — MVP Phase
Audience	Engineering, QA, Product Management	**Classification**	Confidential — Internal Only

Table of Contents

Section	Title	Coverage
1	Product Overview	App purpose, value props, tech stack, architecture
2	Database Schema	All 9 tables with columns, types, constraints, relationships
3	User Roles & Screens	Role definitions, screen inventory, permission matrix
4	Week Lifecycle	5-state machine, transition rules, trigger conditions
5	CP-SAT Solver	Solver config, enforced constraints, fairness objective, output schema
6	API Endpoints	All 20+ REST routes with method, path, auth level, description
7	MVP Status	Completed features and gaps with severity ratings
8	MVP Roadmap	4-phase implementation plan with task-level detail
9	Appendix	Glossary, file structure reference

1. PRODUCT OVERVIEW

What PlanLi is and why it exists

PlanLi is an AI-powered medical shift scheduling system designed for hospital and clinic environments. It replaces manual roster creation with an automated pipeline: staff submit their availability in natural-language Hebrew chat, an AI extracts structured constraints, and a CP-SAT optimization solver (Google OR-Tools) generates fair, coverage-compliant schedules.

The system is admin-driven through a weekly lifecycle: admins create weeks, open constraint collection, trigger the solver, review multiple solver versions, and publish the final schedule for staff to view. The entire interface is in **Hebrew (RTL)** targeting Israeli medical teams.

Core Value Propositions

Capability	Description
Natural Language Input	Staff write Hebrew messages — no forms, no rigid UI. GPT-4o-mini extracts constraints.
Optimal Scheduling	CP-SAT solver ensures all shifts meet min doctor/nurse coverage with fairness objectives.
Hard + Soft Constraints	HARD constraints block assignment; SOFT constraints add penalty to objective function.
Week Lifecycle	Structured DRAFT → COLLECTING → LOCKED → SOLVED → PUBLISHED state machine.
Version Control	Multiple solver runs per week produce versioned schedules; admin picks the best.
Role-Based Access	Admins manage everything; doctors/nurses see only their own data.

Technology Stack

Layer	Technology	Purpose
Frontend	`React 19, TypeScript, Vite, Tailwind CSS 4, React Router 7`	SPA — all user interfaces
Backend API	`Node.js, Express 5, TypeScript, pg, Zod`	REST API, auth middleware, DB queries
Solver Service	`Python 3, Flask, Google OR-Tools CP-SAT`	Constraint optimization engine
Database	`PostgreSQL (Supabase-compatible), 13 migrations`	Primary data store
Auth	`Firebase Auth (client) + Firebase Admin SDK (server)`	Email/password sign-in, JWT verification
LLM	`OpenAI GPT-4o-mini, structured JSON output`	Constraint extraction from Hebrew chat
Locale	`Hebrew (RTL), Asia/Jerusalem timezone`	All UI text, date/time handling

System Architecture

Three independent services communicate over HTTP. The React SPA calls the Node.js API for all data operations. The Node.js API calls the Python Flask solver service only when a schedule solve is triggered. PostgreSQL is the

single source of truth.

Service	Port	Responsibilities	Communicates With
`React Frontend`	`:5173 (dev)`	All UI screens, Firebase Auth client, API calls	Node.js API (REST)
`Node.js API`	`:3000`	Auth middleware, CRUD routes, solver trigger, DB queries	PostgreSQL, Python Solver, Firebase Admin
`Python Solver`	`:5001`	POST /solve — run CP-SAT, return assignments + metadata	Called by Node.js API only
`PostgreSQL`	`:5432`	13 migrations, all persistent data storage	Node.js API only

2. DATABASE SCHEMA

9 core tables — PostgreSQL with 13 migrations

The schema is managed via numbered migration files. All tables use UUID primary keys except **users** which uses Firebase UID (VARCHAR 128). Timestamps are TIMESTAMPTZ where timezone matters (shift times, deadlines) and plain TIMESTAMP for audit fields.

Schema Overview

Table	PK Type	Key Relationships	Purpose
`users`	`VARCHAR(128)`	Root entity — referenced by most tables	Staff profiles with role (doctor/nurse/admin)
`conversations`	`UUID`	`user_id → users`	Chat conversation containers per user
`messages`	`UUID`	`conversation_id → conversations`	Individual chat messages with intent classification
`constraints`	`UUID`	`user_id → users, week_start → weeks`	Extracted scheduling constraints (HARD/SOFT)
`weeks`	`UUID`	UNIQUE on week_start date	Week lifecycle state machine entity
`shifts`	`UUID`	`week_id → weeks`	Shift templates with staffing requirements
`schedule_runs`	`UUID`	`week_start → weeks`	Solver job queue and execution tracking
`schedule_versions`	`UUID`	`week_start → weeks`	Versioned solver outputs with metadata
`schedule_assignments`	`UUID`	`version_id, shift_id, user_id → respective tables`	Final staff-to-shift assignments

Table: users

Firebase-backed user profiles. The id column stores the Firebase UID directly.

Column	Type	Constraint	Description
`id`	`VARCHAR(128)`	PK	Firebase UID
`email`	`VARCHAR(255)`	UNIQUE NOT NULL	Login email address
`full_name`	`VARCHAR(255)`	NOT NULL	Display name (Hebrew supported)
`role`	`VARCHAR(50)`	NOT NULL	'doctor', 'nurse', or 'admin'
`phone`	`VARCHAR(20)`	NULLABLE	Contact phone number
`created_at`	`TIMESTAMP`	DEFAULT NOW()	Record creation time
`updated_at`	`TIMESTAMP`	DEFAULT NOW()	Last modification time

Table: conversations

Container for a user's chat session. Each user can have multiple conversations.

Column	Type	Constraint	Description
id	UUID	PK DEFAULT gen_random_uuid()	Conversation identifier
user_id	VARCHAR(128)	FK → users(id)	Owner of the conversation
message_count	INTEGER	DEFAULT 0	Cached count for display
created_at	TIMESTAMP	DEFAULT NOW()	When conversation started
updated_at	TIMESTAMP	DEFAULT NOW()	Last message time

Table: messages

Individual chat turns. Intent is classified by GPT-4o-mini during extraction.

Column	Type	Constraint	Description
id	UUID	PK	Message identifier
conversation_id	UUID	FK → conversations(id)	Parent conversation
role	TEXT	NOT NULL	'user' or 'assistant'
content	TEXT	NOT NULL	Raw message text (Hebrew)
intent	TEXT	NULLABLE	'constraint', 'schedule_query', 'general', 'out_of_scope'
created_at	TIMESTAMP	DEFAULT NOW()	Message timestamp

Table: constraints

Structured constraints extracted from chat. HARD constraints block assignments; SOFT add objective penalties.

Column	Type	Constraint	Description
id	UUID	PK	Constraint identifier
user_id	VARCHAR	FK → users(id)	Staff member this constraint belongs to
week_start	DATE	FK → weeks(week_start)	Which week this applies to
constraint_type	TEXT	NOT NULL	'HARD' or 'SOFT'
start_time	TIMESTAMPTZ	NULLABLE	Unavailability window start
end_time	TIMESTAMPTZ	NULLABLE	Unavailability window end
is_full_day	BOOLEAN	DEFAULT FALSE	True if entire day is unavailable
reason	TEXT	NULLABLE	Free-text reason from staff (Hebrew)
created_at	TIMESTAMP	DEFAULT NOW()	When constraint was submitted

Table: weeks

The central scheduling unit. Each week progresses through a fixed state machine.

Column	Type	Constraint	Description
id	UUID	PK	Week identifier
week_start	DATE	UNIQUE NOT NULL	Sunday of the work week (Israel)
status	TEXT	NOT NULL	DRAFT | COLLECTING | LOCKED | SOLVED | PUBLISHED
collection_deadline	TIMESTAMPTZ	NULLABLE	Deadline for constraint submission
created_at	TIMESTAMP	DEFAULT NOW()	Week record created
updated_at	TIMESTAMP	DEFAULT NOW()	Last status change

Table: shifts

Shift templates defined by the admin for a given week. The solver fills each shift.

Column	Type	Constraint	Description
id	UUID	PK	Shift identifier
week_id	UUID	FK → weeks(id)	Parent week
week_start	DATE	NOT NULL	Denormalized for query performance
date	DATE	NOT NULL	Specific date of this shift
start_time	TIMESTAMPTZ	NOT NULL	Shift start (with timezone)
end_time	TIMESTAMPTZ	NOT NULL	Shift end (with timezone)
min_doctors	INTEGER	DEFAULT 1	Minimum doctors required
min_nurses	INTEGER	DEFAULT 1	Minimum nurses required
created_at	TIMESTAMP	DEFAULT NOW()	Record creation

Table: schedule_runs

Tracks solver job execution. Fire-and-forget with status polling.

Column	Type	Constraint	Description
id	UUID	PK	Run identifier
week_start	DATE	FK → weeks(week_start)	Target week
status	TEXT	NOT NULL	QUEUED | RUNNING | SUCCEEDED | FAILED
error_message	TEXT	NULLABLE	Error details if FAILED
started_at	TIMESTAMP	NULLABLE	When solver began execution
completed_at	TIMESTAMP	NULLABLE	When solver finished
created_at	TIMESTAMP	DEFAULT NOW()	Job creation time

Table: schedule_versions

Each solver run produces a versioned result. Admin picks which version to publish.

Column	Type	Constraint	Description
id	UUID	PK	Version identifier
week_start	DATE	FK → weeks(week_start)	Target week
version_number	INTEGER	NOT NULL	Increments per week (1, 2, 3…)
is_published	BOOLEAN	DEFAULT FALSE	True when admin publishes this version
published_at	TIMESTAMP	NULLABLE	When this version was published
solver_meta	JSONB	NULLABLE	{solver_status, objective_value, soft_violations, wall_time_seconds}
created_at	TIMESTAMP	DEFAULT NOW()	Version creation time
UNIQUE	(week_start, version_number)		One version number per week

Table: schedule_assignments

The final output: which staff member is assigned to which shift in which version.

Column	Type	Constraint	Description
`id`	`UUID`	PK	Assignment identifier
`schedule_version_id`	`UUID`	FK → schedule_versions(id)	Parent version
`shift_id`	`UUID`	FK → shifts(id)	Target shift
`user_id`	`VARCHAR`	FK → users(id)	Assigned staff member
`source`	`TEXT`	NOT NULL	'solver' (auto) or 'manual' (admin override)
`created_at`	`TIMESTAMP`	DEFAULT NOW()	Assignment creation time
`UNIQUE`	`(version_id, shift_id, user_id)`		No duplicate assignments

3. USER ROLES & SCREENS

Access control and screen inventory

PlanLi has three roles: **Admin**, **Doctor**, and **Nurse**. Admins have full system access; doctors and nurses are restricted to their own data. Role is stored in the users table and enforced by backend middleware on every API request.

Role	Route	Screen Name	Key Functionality
All	`/login`	Login	Firebase email/password sign-in. Redirects by role after auth.
Admin	`/shifts`	Shift Management	Create weeks, add/delete shifts, set collection deadline, open collection, trigger solver.
Admin	`/users`	User Management	Create, edit, delete staff profiles. View role, email, phone.
Admin	`/review`	Schedule Review	View all solver versions with metadata (violations, time, objective). Publish selected version.
Doctor / Nurse	`/chat`	Constraint Chat	Send Hebrew free-text messages. AI extracts constraints and confirms back to user.
Doctor / Nurse	`/schedule`	My Schedule	View personal published shift assignments. Calendar/list view.

Role Permission Matrix

Permission	Admin	Doctor	Nurse
Create / delete weeks	YES	NO	NO
Add / delete shifts	YES	NO	NO
Open constraint collection	YES	NO	NO
Force-lock week	YES	NO	NO
Trigger solver	YES	NO	NO
Review solver versions	YES	NO	NO
Publish schedule	YES	NO	NO
Manage staff (CRUD)	YES	NO	NO
Submit constraints via chat	NO	YES	YES
View own schedule	NO	YES	YES
View all users (read)	YES	NO	NO

4. WEEK LIFECYCLE

State machine governing every scheduling cycle

Every scheduled week progresses through exactly five states. The state machine is enforced in the backend — invalid transitions are rejected with HTTP 400. The lifecycle ensures constraints are only collected during the COLLECTING window and the solver only runs on LOCKED weeks.

State	Who Triggers	Entry Condition	What Happens	Next State
DRAFT	Admin (week creation)	Admin POSTs new week	Week exists, no shifts required yet. Admin adds shifts.	COLLECTING
COLLECTING	Admin action	Admin opens collection with deadline	Staff can now submit constraints via chat. Deadline displayed.	LOCKED
LOCKED	Auto (deadline) or Admin (force)	Deadline passed OR admin force-locks	No more constraint submissions accepted. Week ready for solving.	SOLVED
SOLVED	Admin triggers solver	Week is LOCKED	CP-SAT solver runs. One or more schedule_versions created. Admin reviews.	PUBLISHED
PUBLISHED	Admin publishes version	At least one SUCCEEDED version exists	Selected version marked is_published=true. Staff can view assignments.	Terminal

State Transition Rules

- DRAFT → COLLECTING requires a collection_deadline to be set.
- COLLECTING → LOCKED happens automatically when collection_deadline passes (server-side check), or admin can force-lock at any time.
- LOCKED → SOLVED is triggered by admin calling POST /api/schedule/solve. The week must be in LOCKED state.
- A week can be re-solved (SOLVED state can have multiple schedule_versions). The week stays in SOLVED after each run.
- SOLVED → PUBLISHED: Admin calls POST /api/schedule/publish with a version ID. Only one version can be published per week.
- There is no rollback state — published schedules are immutable. Manual overrides create new 'manual' source assignments.

5. CP-SAT SOLVER

The optimization engine — Python + Google OR-Tools

The solver is a standalone Python Flask microservice. It receives a self-contained payload from the Node.js API and returns assignments with metadata. It has no direct database access — all data is passed in the request body.

Solver Configuration

Parameter	Value	Purpose
Solver	`Google OR-Tools CP-SAT`	Constraint Programming — Satisfiability solver
Time limit	`20 seconds`	Hard wall-clock timeout per solve run
Workers	`8 threads`	Parallel search workers for faster solve
Decision variables	`x[user_id, shift_id] ∈ {0, 1}`	Binary: 1 = user assigned to shift
Input endpoint	`POST /solve`	Accepts JSON payload from Node.js API
Output	`{status, assignments[], solver_meta{}}`	Returned to Node.js for DB persistence

Enforced Constraints

Constraint Type	Category	Description
Minimum coverage	HARD	Each shift must have ≥ min_doctors doctors AND ≥ min_nurses nurses assigned.
No overlap	HARD	A user cannot be assigned to two shifts with overlapping time windows.
HARD unavailability	HARD	If a user has a HARD constraint covering a shift's window, they cannot be assigned.
SOFT unavailability	SOFT	SOFT constraint violations add a penalty term to the objective function — they don't block assignment.
Fairness — max-min balance	Objective	Minimize the difference between the maximum and minimum number of shifts assigned across all users of the same role.

solver_meta JSON Schema

The solver returns a metadata object stored in schedule_versions.solver_meta (JSONB):

Field	Type	Description
`solver_status`	`string`	OR-Tools status: OPTIMAL, FEASIBLE, INFEASIBLE, UNKNOWN
`objective_value`	`number`	Final objective score (lower = better; 0 = perfect fairness, no soft violations)
`soft_violations`	`number`	Count of SOFT constraints that were violated in this solution
`wall_time_seconds`	`number`	Actual solver wall-clock time in seconds (max 20s)

6. API ENDPOINTS

20+ REST routes — Node.js / Express 5

All endpoints are prefixed with **/api**. Authentication is handled via Firebase ID token in the Authorization header (Bearer token). The backend middleware verifies the token with Firebase Admin SDK and attaches the user record to the request.

Auth & User Management

Method	Route	Auth	Description
POST	`/api/users/sync`	All	Sync Firebase user to PostgreSQL after login. Creates user record if not exists.
GET	`/api/users/me`	All	Return current authenticated user's profile (id, email, name, role).
GET	`/api/users`	Admin	List all users with role, email, phone. Paginated.
POST	`/api/users`	Admin	Create new staff user. Body: {email, full_name, role, phone}.
PUT	`/api/users/:id`	Admin	Update user fields. Partial update supported.
DELETE	`/api/users/:id`	Admin	Soft-delete user record. Cannot delete self.

Weeks & Shifts

Method	Route	Auth	Description
GET	`/api/shifts/weeks`	Admin	List all weeks with status, deadline, shift count.
POST	`/api/shifts/weeks`	Admin	Create a new week. Body: {week_start: 'YYYY-MM-DD'}.
POST	`/api/shifts/weeks/:weekStart/open-collection`	Admin	Transition week DRAFT→COLLECTING. Body: {collection_deadline}.
POST	`/api/shifts/weeks/:weekStart/lock`	Admin	Force-lock week regardless of deadline. COLLECTING→LOCKED.
GET	`/api/shifts/weeks/:weekStart/shifts`	Admin	List all shifts for a specific week.
POST	`/api/shifts/shifts`	Admin	Create shift. Body: {week_start, date, start_time, end_time, min_doctors, min_nurses}.
DELETE	`/api/shifts/shifts/:id`	Admin	Delete a shift (only if week is DRAFT or COLLECTING).

Chat & Constraints

Method	Route	Auth	Description
POST	/api/chat/message	Doctor/Nurse	Send chat message. LLM classifies intent, extracts constraints, saves message + constraint records.
GET	/api/chat/constraints	Doctor/Nurse	Return all constraints for current user, optionally filtered by week_start.

Schedule & Solver

Method	Route	Auth	Description
POST	/api/schedule/solve	Admin	Trigger CP-SAT solver for a week. Body: {week_start}. Creates schedule_run record, calls Python service.
GET	/api/schedule/status/:weekStart	Admin	Poll solver run status (QUEUED/RUNNING/SUCCEEDED/FAILED) with error details.
GET	/api/schedule/versions/:weekStart	Admin	List all schedule_versions for a week with solver_meta and published status.
POST	/api/schedule/publish	Admin	Publish a specific version. Body: {version_id}. Sets is_published=true, week→PUBLISHED.
GET	/api/schedule/my-schedule	Doctor/Nurse	Return published shift assignments for the current user across all weeks.

7. MVP STATUS

What's working, what needs fixing

The following table captures the current completion state of every major feature area. Green = working and tested. Orange = implemented but broken or incomplete. Red = not started.

Completed Features

Feature Area	Status	Notes
Firebase Auth	DONE	Login, logout, session persistence, token refresh all working.
User Management CRUD	DONE	Admin can create, edit, delete staff. Role assignment works.
Week Creation & Shift Management	DONE	Admin can create weeks, add shifts with times and staffing requirements.
LLM Constraint Extraction	DONE	GPT-4o-mini correctly extracts constraints from Hebrew chat with structured JSON output.
CP-SAT Solver (Python)	DONE	Solver logic implemented: coverage, no-overlap, hard/soft constraints, fairness objective.
Schedule Versioning & Publishing	DONE	DB schema and backend logic for versioning and publishing complete.
Database Migrations	DONE	13 migrations applied. Schema stable.
Mobile Sidebar (hamburger)	DONE	Basic hamburger menu implemented. Navigation works on mobile.

MVP Gaps — Needs Work

#	Gap	Severity	Description
1	Solver UI Flow Broken	CRITICAL	Trigger button in /shifts, version list in /review, and publish action need fixing. Solver API integration may have bugs.
2	Mobile Responsiveness	CRITICAL	ShiftPage, ScheduleReviewPage, MySchedulePage, UserManagement are not properly responsive. Full mobile overhaul needed.
3	Schedule Query in Chat	IMPORTANT	Intent 'schedule_query' is classified but not executed. Staff can't ask the chatbot to show their schedule.
4	UI Polish	IMPORTANT	Tables are functional but not beautiful. Needs modern card-based design, consistent color system, and typography.
5	Loading / Error States	IMPORTANT	Missing skeletons, error boundaries, and toast notifications. Users see blank screens on slow loads.
6	Solver Status Polling	NICE-TO-HAVE	Solve is fire-and-forget with no real-time status updates in the UI. Admin must manually refresh.

8. MVP ROADMAP

Prioritized implementation plan — 4 phases

Phase 1 — Fix Core Solver Flow — CRITICAL — Do First

Task	Item	Implementation Detail
1.1	Fix solve trigger button in /shifts admin panel	The 'Solve' button should POST /api/schedule/solve and show feedback.
1.2	Fix /review page — show solver versions with metadata	Display version_number, solver_status, objective_value, soft_violations, wall_time_seconds in a table.
1.3	Fix publish action and confirmation modal	Admin selects version, confirms dialog, POST /api/schedule/publish, week transitions to PUBLISHED.
1.4	Add polling for solver run status	Poll GET /api/schedule/status/:weekStart every 2s. Show QUEUED → RUNNING → SUCCEEDED/FAILED with spinner.
1.5	Show solver errors clearly in the UI	If status = FAILED, display error_message in a red alert card.

Phase 2 — Mobile Responsiveness — CRITICAL — High User Impact

Task	Item	Implementation Detail
2.1	Full responsive overhaul of all pages	Use Tailwind responsive breakpoints. Test on 375px, 768px, 1280px.
2.2	Bottom navigation bar for mobile (staff role)	Fixed bottom nav with Chat and Schedule icons for doctor/nurse on mobile.
2.3	Admin pages: stack layouts vertically on mobile	ShiftPage and UserManagement tables become stacked cards on small screens.
2.4	Chat interface: proper keyboard handling on iOS/Android	Viewport resize on keyboard open. Input stays visible. Scroll to bottom on new message.
2.5	Schedule view: card-based layout on mobile	Replace table with shift cards showing date, time, role on MySchedulePage.

Phase 3 — UI Polish — IMPORTANT — Quality of Life

Task	Item	Implementation Detail
3.1	Replace raw HTML tables with styled card grids	Use card components with shadows, rounded corners, and consistent spacing.
3.2	Add loading skeletons for all data fetches	Every list/table shows skeleton placeholder rows during loading.
3.3	Add toast notifications for actions	Success/error toasts on save, delete, publish, solve trigger.
3.4	Consistent color system and typography	Define Tailwind theme tokens for navy, blue, grey. Apply consistently.
3.5	Week status badge chips (colored by state)	DRAFT=grey, COLLECTING=blue, LOCKED=orange, SOLVED=purple, PUBLISHED=green.

Task	Item	Implementation Detail
3.6	Empty states with helpful CTAs	When no shifts/users/constraints exist, show icon + message + action button.

Phase 4 — Chat Enhancement — NICE TO HAVE — Post-MVP

Task	Item	Implementation Detail
4.1	Execute schedule_query intent from published data	When intent=schedule_query, fetch published assignments and return formatted Hebrew response.
4.2	Show constraint confirmation cards in chat	After extracting a constraint, render a structured card (type, date, time) instead of plain text.
4.3	Allow constraint deletion via chat	If user says 'cancel my request for Tuesday', find and delete matching constraint.

9. APPENDIX

Glossary and quick-reference

Glossary

Term	Definition
CP-SAT	Constraint Programming — Satisfiability. Google OR-Tools solver that treats scheduling as a combinatorial optimization problem with hard/soft constraints.
HARD Constraint	An absolute restriction. If violated, the solver cannot produce a valid solution. Example: user is unavailable for surgery on Tuesday.
SOFT Constraint	A preference. Violation adds penalty points to the objective function. The solver tries to honor it but may not if coverage requires it.
Fairness Objective	Minimize (max_shifts_assigned - min_shifts_assigned) across all users of the same role. Prevents one nurse getting all the work.
Week Lifecycle	The 5-state machine (DRAFT → COLLECTING → LOCKED → SOLVED → PUBLISHED) that governs every scheduling cycle.
Firebase UID	Unique user identifier issued by Firebase Authentication. Used as the primary key in the users table.
solver_meta	JSONB field in schedule_versions storing solver runtime data: status, objective value, soft violations, and wall time.
RTL	Right-to-Left. The text direction for Hebrew. Affects CSS layout, text alignment, and component mirroring throughout the UI.
schedule_run	A DB record tracking a single solver execution: creation, queued, running, succeeded/failed states with timestamps.

File Structure Reference

Key directories and files in the monorepo:

Path	Contents
`frontend/src/pages/`	React page components (ShiftsPage, ReviewPage, ChatPage, MySchedulePage, UserManagement, Login)
`frontend/src/components/`	Shared UI components: Sidebar, Header, modals, cards
`backend/src/routes/`	Express route handlers: users.ts, shifts.ts, chat.ts, schedule.ts
`backend/src/middleware/`	Firebase auth middleware, error handler, Zod validation
`backend/migrations/`	13 numbered SQL migration files (001_create_users.sql … 013_…)
`solver/`	Python Flask solver service: app.py, solver.py, requirements.txt
`.env (backend)`	DATABASE_URL, FIREBASE_*, OPENAI_API_KEY, SOLVER_URL
`.env (frontend)`	VITE_API_URL, VITE_FIREBASE_* config keys